

Machine Learning for Credit Default Prediction on LendingClub's P2P Lending Platform

Business Analytics using Python (EM5023)

B.S.P. Krishna Chaitanya Reddy	AI23BTECH11002
B. Manoj Kumar	MS23BTECH11007
S. Sukesh Kumar	AI23BTECH11024
V. Akshay Kumar	AI23BTECH11030

IIT Hyderabad, May 2025

Contents

1	Introduction	2
2	Data Description	2
2.1	Source and Filtering	2
2.2	Variables	3
2.3	Class Imbalance	3
3	Data Cleaning & Preprocessing	3
4	Exploratory Data Analysis	4
4.1	Correlation Structure	4
4.2	Grade, Purpose, and FICO Effects	4
4.3	Interaction Effects	6
4.4	Temporal and Credit History Patterns	6
5	Dimensionality Reduction	7
5.1	VIF Filter	7
5.2	Lasso Feature Selection	8
5.3	PCA (Exploration Only)	8
6	Models & Methodology	8
6.1	SMOTE Inside CV Pipeline	8
6.2	Model Descriptions	9
6.3	Hyperparameter Tuning	9
7	Evaluation & Best Model Selection	10
7.1	Learning Curves	10
7.2	Model Comparison	10
7.3	Grade vs ML: Can We Beat LendingClub?	12
7.4	Threshold Optimisation	12
7.5	Feature Importance	13
8	Portfolio Risk Profiling & Active Loan Scoring	14
8.1	Risk Tier Validation on Test Set	14
8.2	Active Loan Risk Profiling	14
9	Actionable Insights	15
10	Conclusion & Limitations	17

Abstract

This report uses machine learning to predict which loans on LendingClub’s platform are likely to default, helping investors and risk teams identify bad loans before losses occur. Starting from 2.26 million raw loan records (2007–2018 Q4), 1,345,310 loans with clear outcomes were retained. Systematic preprocessing reduced 151 raw variables to 99 engineered features, followed by VIF-based multicollinearity filtering (89 features) and Lasso regularisation (77 features). A critical methodological decision was to apply SMOTE oversampling *inside* each cross-validation fold via `imblearn.pipeline`, preventing synthetic data leakage that inflated CV scores by approximately 0.14 AUC in initial experiments.

Six models: Logistic Regression, LDA, Decision Tree, Random Forest, XGBoost, and SVM were trained and evaluated. XGBoost achieved the highest AUC-ROC of 0.7330, representing a 7.8% relative improvement over LendingClub’s internal grade alone (AUC 0.6801). Threshold optimisation revealed that the default 0.50 cutoff yields only 10.8% defaulter recall; lowering to 0.23 (F1-optimal) increases recall to 62.0%. The model was deployed to score 912,569 active loans, identifying 32.1% as high-risk.

Key takeaway for decision makers: The single most impactful operational change is just lowering the decision threshold from 0.50 to 0.23, which raises bad loans caught from 1 in 10 to 6 in 10. One third of currently active loans show elevated default risk, and the model catches risks that LendingClub’s own grading system misses, suggesting it works best as a second check on top of existing processes.

1 Introduction

Credit default prediction is critical for financial risk management. The global consumer lending market exceeds \$10 trillion, meaning even small improvements in detecting defaults can prevent significant losses. LendingClub, a major peer-to-peer lending platform, originated over \$50 billion in loans and publicly released its data, providing a real-world dataset for building and testing credit risk models. Unlike traditional bank lending, peer-to-peer platforms connect real investors directly to borrowers, meaning poor risk assessment leads to direct investor losses rather than bank writeoffs.

This analysis addresses four questions: **(1) Which machine learning method best predicts loan default? (2) Which borrower characteristics are the strongest risk drivers? (3) Can a machine learning model outperform LendingClub’s own grading system? (4) What is the risk profile of currently active loans?** Starting from 2.26 million raw records and 151 variables, we built a complete pipeline covering data cleaning, exploratory analysis, dimensionality reduction using VIF and Lasso, SMOTE-based class balancing inside cross-validation folds, and evaluation of six models ranging from simple linear classifiers to ensemble methods.

2 Data Description

2.1 Source and Filtering

The dataset comes from LendingClub’s publicly released loan data (2007–2018 Q4), available on Kaggle [1]. Rejected loan applications were excluded because they do not have a known outcome. From 2,260,701 raw records, we kept only loans with clear outcomes,

either “Fully Paid” (1,076,751) or “Charged Off” (268,559), giving **1,345,310 usable observations**. Loans still in progress (“Current,” “Late,” “In Grace Period”) were set aside as a 912,569 record active scoring set, used only in Section 8.2. Because the model only sees loans that LendingClub already approved, it cannot directly score new, not approved applications, a limitation known as **survivorship bias**.

2.2 Variables

We removed 17 variables that are only known after a loan is issued (such as `total_pymnt` and `recoveries`), 20 columns that are mostly empty, 5 ID and text columns, 4 categorical columns that could leak future information, and 2 raw date columns after feature engineering. This left 84 columns. We created `credit_hist_months` from `issue_d` and `earliest_cr_line` to capture how long a borrower has had credit history. Three variables were encoded as ordered ranks: `term` (36 months becomes 0, 60 months becomes 1), `grade` (A becomes 7 down to G as 1), and `emp_length` (0 to 10 years). Five category variables were converted to binary columns using one-hot encoding, adding 23 new columns. The final dataset had **99 features** in total (76 numeric + 23 binary).

2.3 Class Imbalance

Figure 1 shows that 80.0% of loans were fully repaid and 20.0% were charged off. If a model simply predicted “always repay,” it would achieve 80% accuracy while catching zero defaulters. This imbalance makes accuracy a misleading metric and is why we use SMOTE to balance the training data and AUC-ROC as the primary evaluation measure.

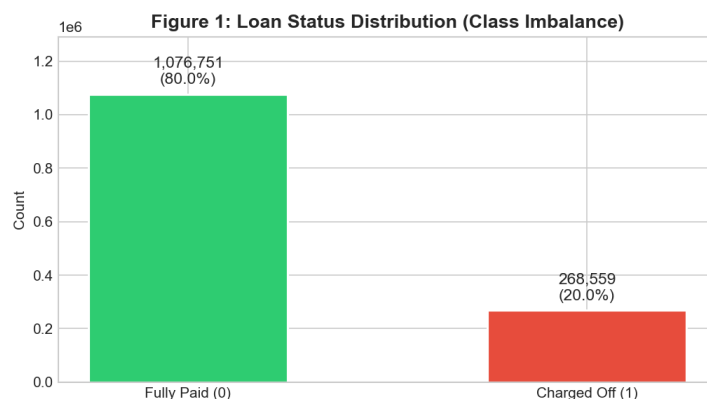


Figure 1: Loan Status Distribution (n = 1,345,310)

3 Data Cleaning & Preprocessing

Missing numeric values were filled with the median value from the training set, and any remaining gaps were set to zero. Six financial variables with extreme high values (such as annual income exceeding \$9 million) were capped at their 99th percentile to prevent them from distorting the models. For example, annual income was limited to \$250,000.

During preprocessing, we discovered and fixed a significant bug. The raw data stored loan term as “ 36 months” with a leading space, which caused the mapping function to fail silently and turn the entire column into zeros. After adding a string strip to remove the

extra space, the term variable regained its true values (324,567 loans at 60 months), and Lasso correctly identified it as the single strongest predictor with a coefficient of 0.110.

The data was split 80/20 using stratified sampling so that both the training set (1,076,248 rows) and test set (269,062 rows) maintain the same 19.96% default rate. Numeric features were standardised to have zero mean and unit variance, while binary features were left unchanged since scaling them has no effect on tree-based models.

4 Exploratory Data Analysis

4.1 Correlation Structure

Figure 2 shows how 15 key numeric features relate to each other. We identified any pair with a correlation above 0.7 as potentially problematic for linear models. Four pairs exceed this threshold: FICO low and FICO high are nearly identical ($r = 1.00$), monthly installment closely tracks loan amount ($r = 0.95$), revolving balance and total revolving limit move together ($r = 0.74$), and open accounts strongly predict total accounts ($r = 0.70$). These overlaps are expected but problematic for linear models, which is why we apply VIF filtering in the next section to remove redundant features.

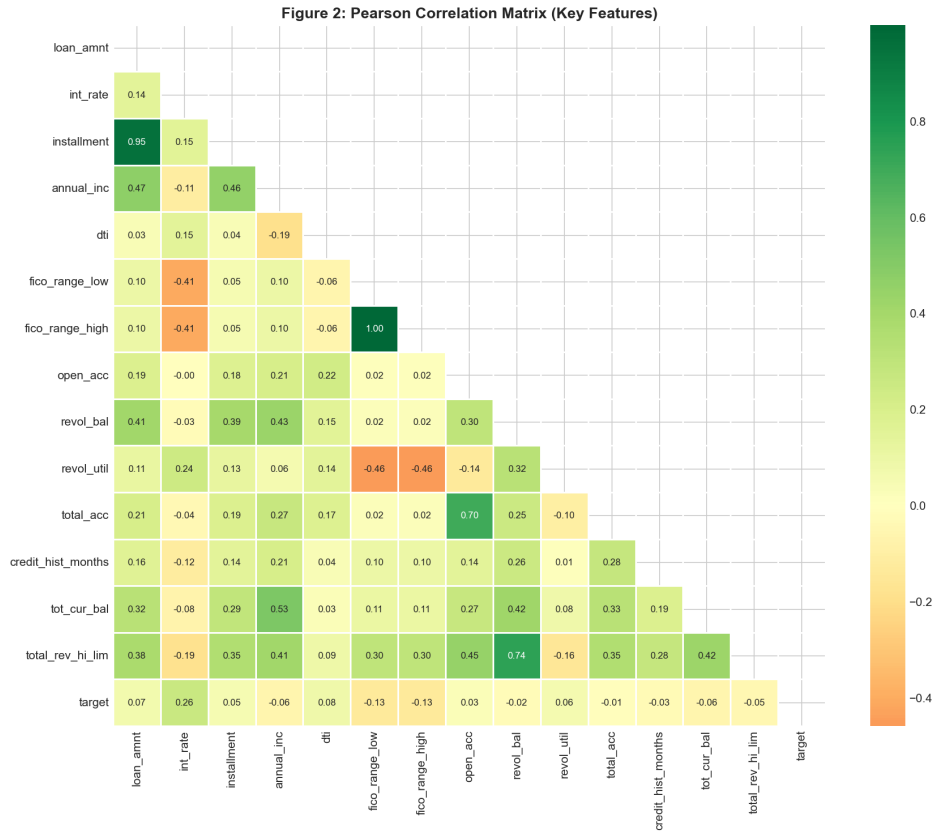


Figure 2: Pearson Correlation Matrix (Key Features)

4.2 Grade, Purpose, and FICO Effects

Figure 3 shows a clear pattern by LendingClub’s internal grade: Grade A loans default at roughly 5%, while Grade G loans default at over 35%. This tenfold difference confirms

that the platform’s own grading system is a strong signal. Figure 4 then breaks down two specific risk factors. The left panel shows that borrowers who defaulted tend to have FICO scores roughly 20 to 30 points lower than those who repaid, a visible shift even in overlapping distributions. The right panel shows that default rates climb steadily as the debt-to-income ratio increases above 20%, confirming DTI as an independent risk signal. Figure 5 places all six features side by side, making it easy to see that defaulters consistently have higher interest rates, lower FICO scores, higher DTI, and more revolving credit usage. The notched boxes show confidence intervals around the medians, and where they do not overlap, we can be reasonably confident the difference between the two groups is real rather than due to chance.

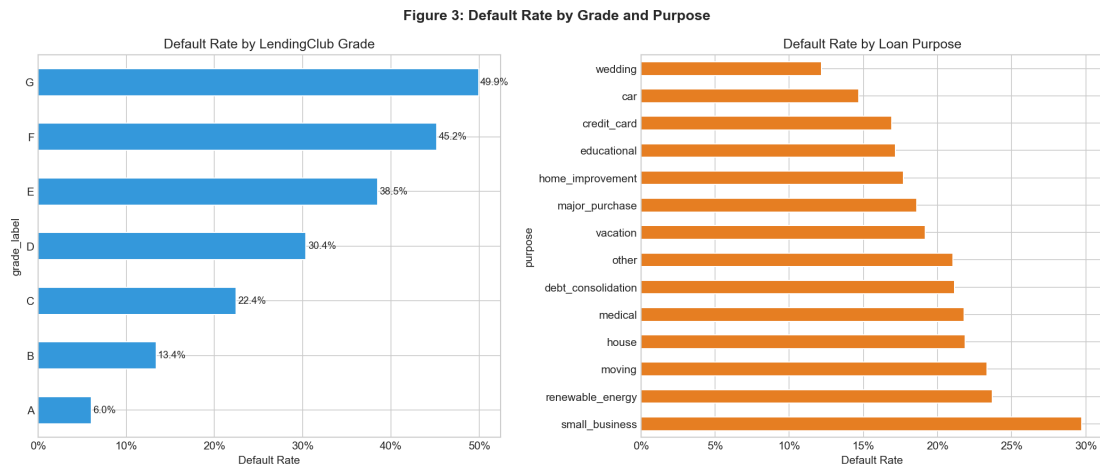


Figure 3: Default Rate by Grade and Loan Purpose

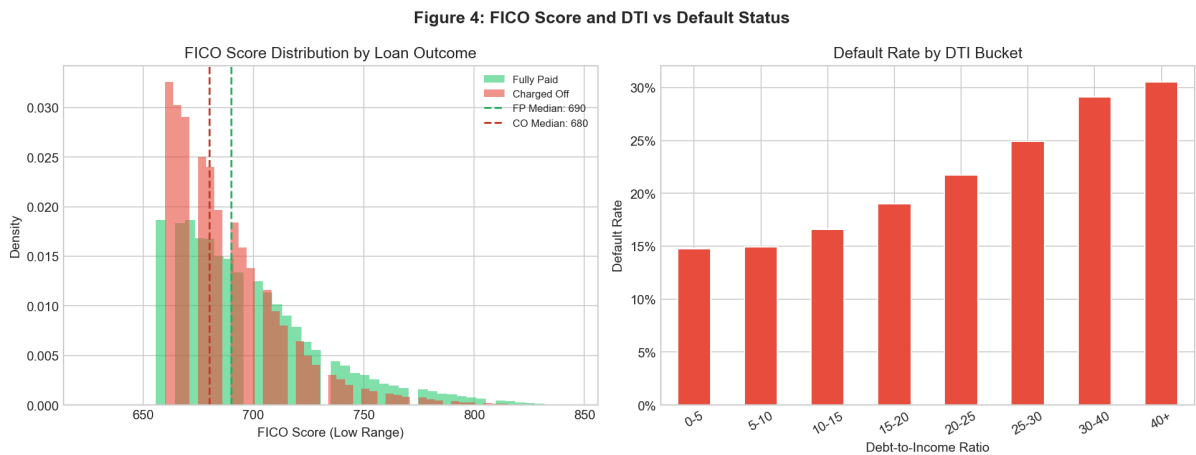


Figure 4: FICO Score Distribution and Default Rate by DTI Bucket

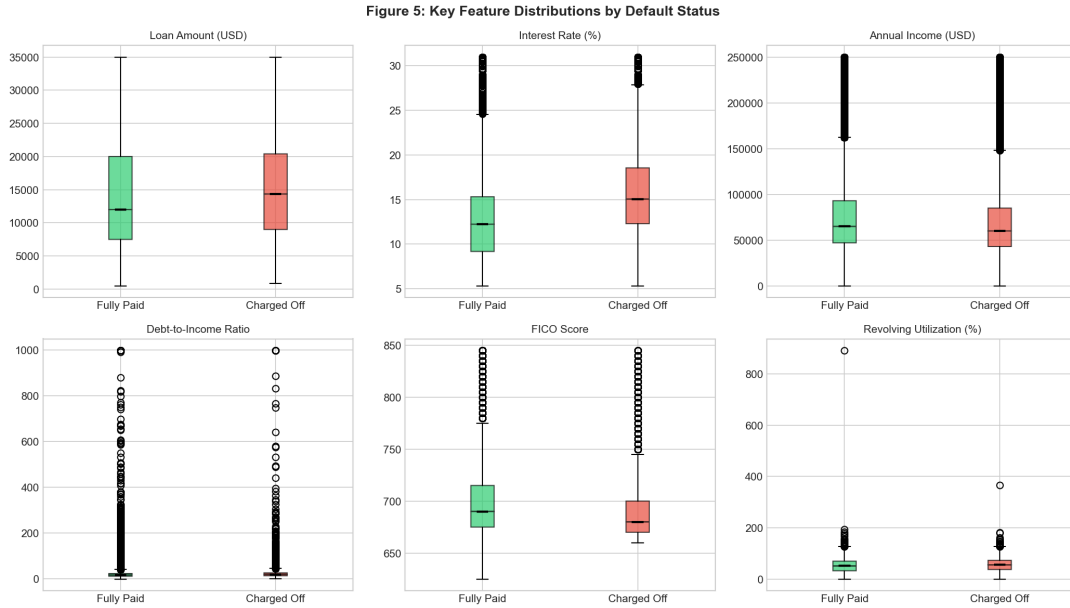


Figure 5: Key Feature Distributions by Default Status

4.3 Interaction Effects

Figure 6 reveals two policy-relevant interactions. The left panel shows that among Grade D borrowers, those who did not provide income verification default at least 5 percentage points more than verified borrowers with the same grade. This suggests that verification itself carries useful information beyond the grade, and unverified loans within a risk grade deserve extra caution. The right panel compares loan terms across home ownership types and shows that 60-month loans consistently default at higher rates than 36-month loans regardless of whether the borrower rents, owns, or has a mortgage. This confirms that longer loan terms genuinely increase risk and should be priced accordingly.

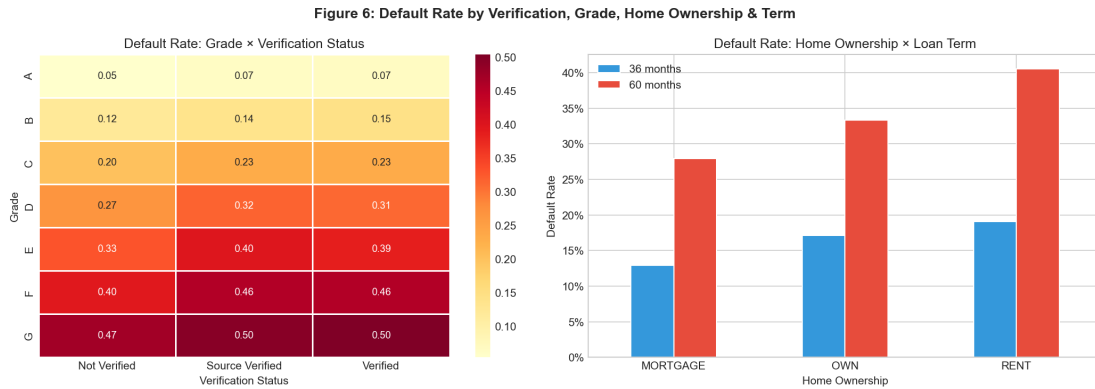


Figure 6: Default Rate by Verification×Grade and Home Ownership×Term

4.4 Temporal and Credit History Patterns

Figure 7 highlights two patterns. First, borrowers with less than 3 years of credit history default more often, which makes sense because a short track record leaves the model with less information to assess risk. Second, loans issued around 2015 and 2016 show

a noticeable jump in default rates compared to earlier years, possibly because lending standards became more relaxed during a period of economic growth. This suggests that the year a loan was originated matters and should not be ignored when evaluating model performance over time.

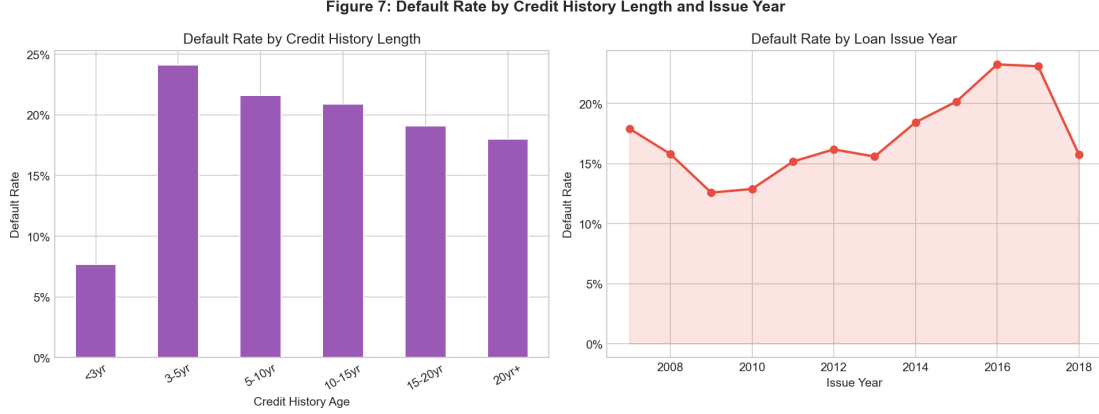


Figure 7: Default Rate by Credit History Length and Issue Year

5 Dimensionality Reduction

5.1 VIF Filter

When two features are highly correlated, a linear model cannot reliably tell which one matters. We used the Variance Inflation Factor to measure this problem and iteratively removed any feature with a VIF above 10. Table 1 lists the 10 features that were dropped. The most extreme case was `fico_range_low` with a VIF over 7 million, because it is essentially a duplicate of `fico_range_high`. One notable removal is `int_rate` (VIF = 11.6), which overlaps heavily with `grade` since LendingClub essentially sets the interest rate based on the grade. Since grade remains in the model, the interest rate signal is preserved without keeping the redundant variable. After filtering, the feature set contains 66 numeric and 23 binary features for a total of **89 features**.

Table 1: Features Removed by Iterative VIF > 10

Feature	VIF	Reason
<code>fico_range_low</code>	7,522,644	Redundant with <code>fico_range_high</code>
<code>num_rev_tl_bal_gt_0</code>	43.5	Correlated with revolving metrics
<code>num_sats</code>	33.2	Correlated with <code>num_bc_sats</code>
<code>total_acc</code>	24.8	Correlated with <code>open_acc</code>
<code>num_op_rev_tl</code>	15.4	Correlated with <code>num_rev_accts</code>
<code>total_rev_hi_lim</code>	13.5	Correlated with <code>revol_bal</code>
<code>total_bc_limit</code>	13.0	Correlated with <code>bc_open_to_buy</code>
<code>loan_amnt</code>	12.4	Correlated with <code>installment</code>
<code>tot_cur_bal</code>	11.9	Correlated with balance metrics
<code>int_rate</code>	11.6	Correlated with <code>grade</code>

5.2 Lasso Feature Selection

LassoCV with 5-fold cross-validation on a 200K subsample found the optimal regularisation strength at $\alpha = 0.000105$ and kept `term` as the top feature with a coefficient of 0.110. This makes business sense because 60-month loans genuinely carry more risk than 36-month loans, and this simple binary variable outperforms complex engineered features in importance. The next strongest predictors were `purpose_small_business` (0.058) and `grade` (-0.056 , confirming that higher grades mean lower risk). We kept all 77 features because LassoCV chose the regularisation level that cross-validation determined was optimal; forcing it to drop more features would mean throwing away real signal just to get a shorter feature list.

5.3 PCA (Exploration Only)

PCA was applied to the 66 VIF-filtered numeric features to see how many components are needed to represent the data well. Figure 8 shows that 45 principal components capture 95% of the information and 58 components capture 99%. This means PCA can compress 66 features down to around 45 without losing much accuracy. However, PCA was not used for any model because each principal component is a mixture of all original features, making it impossible to explain why a specific loan was approved or rejected. Since the goal of this analysis is to provide actionable business insights, we kept the Lasso-selected features instead.

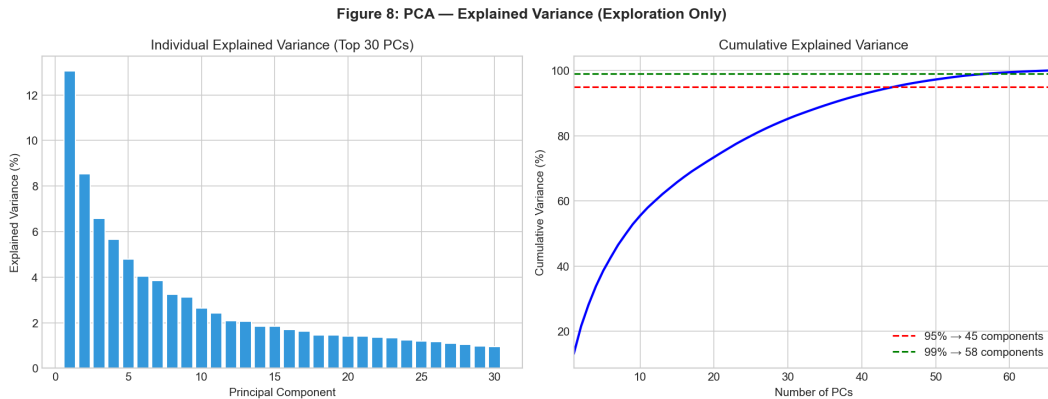


Figure 8: PCA Explained Variance (95% at 45 Components, 99% at 58)

6 Models & Methodology

6.1 SMOTE Inside CV Pipeline

The dataset has an 80/20 class imbalance (80% repaid, 20% defaulted). To handle this, we use SMOTE to generate synthetic default examples to balance the training data. A key question is where exactly SMOTE should happen. In our first attempt, we applied SMOTE to the entire training set before splitting into folds. This means synthetic samples leaked into validation folds and made model performance look better than it really is. For example, Random Forest showed a CV AUC of 0.856 but a test AUC of only 0.714, a gap of 0.14 that is entirely artificial.

We fixed this by placing SMOTE inside each cross-validation fold using an `imblearn.pipeline.Pipeline`. Now, each fold only trains on real data plus synthetic samples generated from that fold alone, and validates on real data only. After this fix, the CV and test AUC values align closely (gaps under 0.005 in Table 3), confirming that our hyperparameter choices are honest. We run GridSearch on a 200K subsample for speed, then refit each model on the full 1.08M training rows using the same pipeline structure.

6.2 Model Descriptions

We compare six models, each taking a different approach to the classification problem. Logistic Regression serves as a simple linear baseline that is easy to interpret. LDA also uses a linear boundary but assumes each class follows a bell-shaped distribution, so we feed it only numeric features to reduce the impact of that assumption being partially wrong. Decision Tree splits the data into simple yes or no rules but can memorise the training data too closely. Random Forest fixes this by averaging many trees built on different subsets of the data. XGBoost goes further by building trees one at a time, where each new tree focuses on correcting the mistakes of the previous ones. SVM finds the widest possible boundary between classes but is too slow for the full dataset, so we use a faster linear version on a 50K subsample. We chose these six because they span the range from simple to complex and from linear to nonlinear, giving a fair picture of what different models can achieve on this data.

6.3 Hyperparameter Tuning

Table 2 summarises the grid search results for each model.

Table 2: Hyperparameter Tuning Summary

Model	CV AUC	Parameters Tested	Best Config
Logistic Regression	0.7184	Regularisation strength: 4 values	$C = 1.0$
LDA	—	None	Default
Decision Tree	0.7011	Depth: 4 values, Split size: 3 values, Criterion: 2 options	depth=10, split=1000, gini
Random Forest	0.7206	Trees: 2 values, Depth: 3 options, Features: 2 options	n=500, depth=20, sqrt
XGBoost	0.7290	Trees: 2 values, Depth: 2 values, Learning rate: 2 values, Subsample: 2 values	n=500, depth=6, lr=0.05, sub=0.8
SVM [†]	0.7161	Regularisation strength: 4 values	$C = 0.01$

[†]SVM: 3-fold CV on 50K subsample. All others: 5-fold on 200K subsample with SMOTE inside pipeline.

7 Evaluation & Best Model Selection

7.1 Learning Curves

Figure 9 shows how model performance changes as we give it more training data, using non-SMOTE data with balanced class weights to keep the evaluation honest. The linear models (Logistic Regression, LDA, SVM) quickly flatten around AUC 0.70 to 0.72 and stop improving, meaning they have learned everything the features can offer. The tree models (Random Forest, XGBoost) show a larger gap between training and validation scores, but their validation AUC is still climbing at the right edge of the plot. This tells us they are not yet saturated and would improve further on the full 1.08M training set. Notably, no model shows severe overfitting, which confirms that the dataset provides enough data for all approaches.

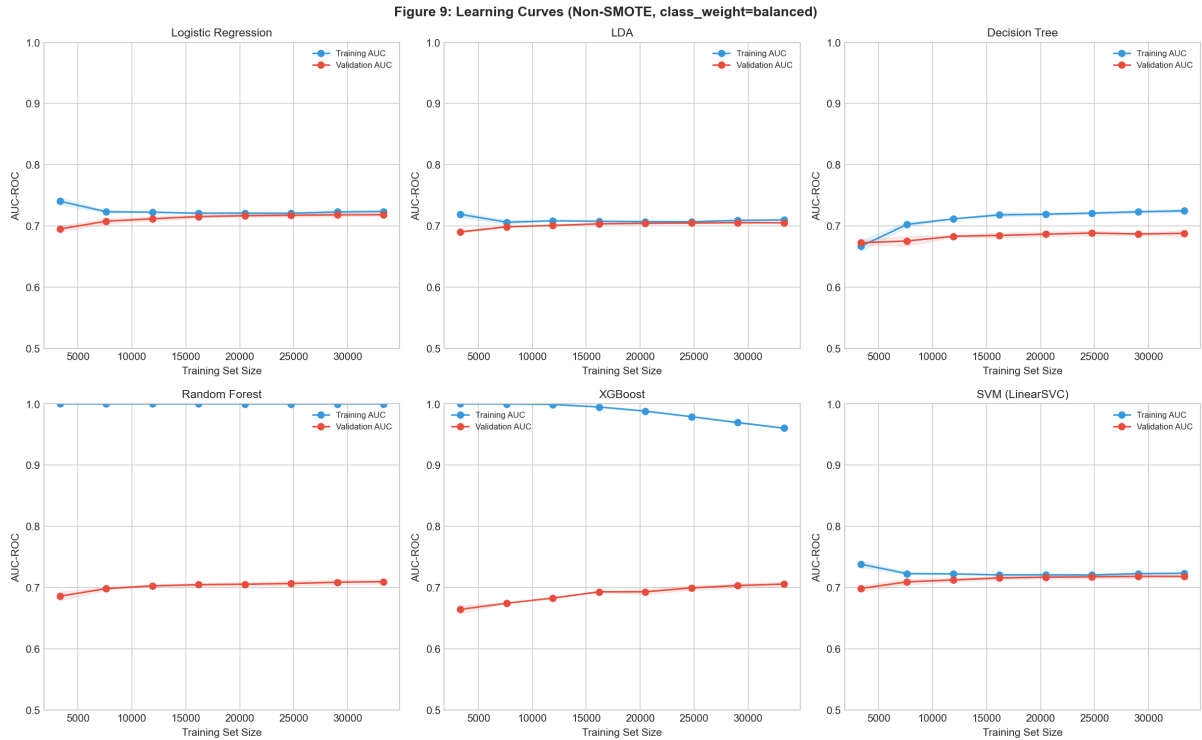


Figure 9: Learning Curves (Non-SMOTE, class_weight=balanced, 50K subsample)

7.2 Model Comparison

Table 3 compares all models on the test set. XGBoost achieves the highest AUC at 0.7330, with Random Forest close behind at 0.7244. The simpler linear models (Logistic Regression and LDA) both settle around 0.705. Decision Tree performs worst among Lasso-selected features at 0.7042. The bottom two rows show what happens when we skip dimensionality reduction entirely and train Random Forest and XGBoost on all 99 original features. The difference is tiny (0.007 and 0.008 AUC), which suggests that tree models already handle redundant features well on their own and gain almost nothing from our VIF and Lasso pipeline.

While ROC curves evaluate overall separability, Precision-Recall (PR) curves are more informative for our 80/20 imbalanced dataset because they focus on how well models

identify the minority class (defaulters) without being skewed by the high true-negative rate. Figure 11 shows the PR curves for all models.

Table 3: Model Performance on Test Set ($n = 269,062$)

Model	CV AUC	Test AUC	Accuracy	Prec. (Def.)	Recall (Def.)	F1 (Def.)
Logistic Regression	0.7184	0.7183	0.7793	0.4306	0.3270	0.3717
LDA	—	0.7046	0.7754	0.4121	0.2937	0.3430
Decision Tree	0.7011	0.7042	0.7973	0.4701	0.1228	0.1947
Random Forest	0.7206	0.7244	0.8033	0.5405	0.0980	0.1660
XGBoost[†]	0.7290	0.7330	0.8068	0.5868	0.1082	0.1827
SVM [†]	0.7161	0.7165	0.7774	0.4268	0.3358	0.3759
<i>Ablation (Raw features, 100K subsample):</i>						
Random Forest (Raw)	—	0.7194	0.8029	0.5343	0.0978	0.1653
XGBoost (Raw)	—	0.7258	0.8049	0.5552	0.1139	0.1891

[†] SVM was trained on 50K rows only. Its AUC of 0.7165 is not directly comparable to other models trained on over 1 million rows and should be treated as a lower bound estimate.

[‡] XGBoost test AUC (0.7330) is slightly above CV AUC (0.7290). This is unusual but is a positive sign, confirming that placing SMOTE inside each fold prevented overfitting and that the model generalises well to unseen data.

Figure 10: ROC Curves — All Models

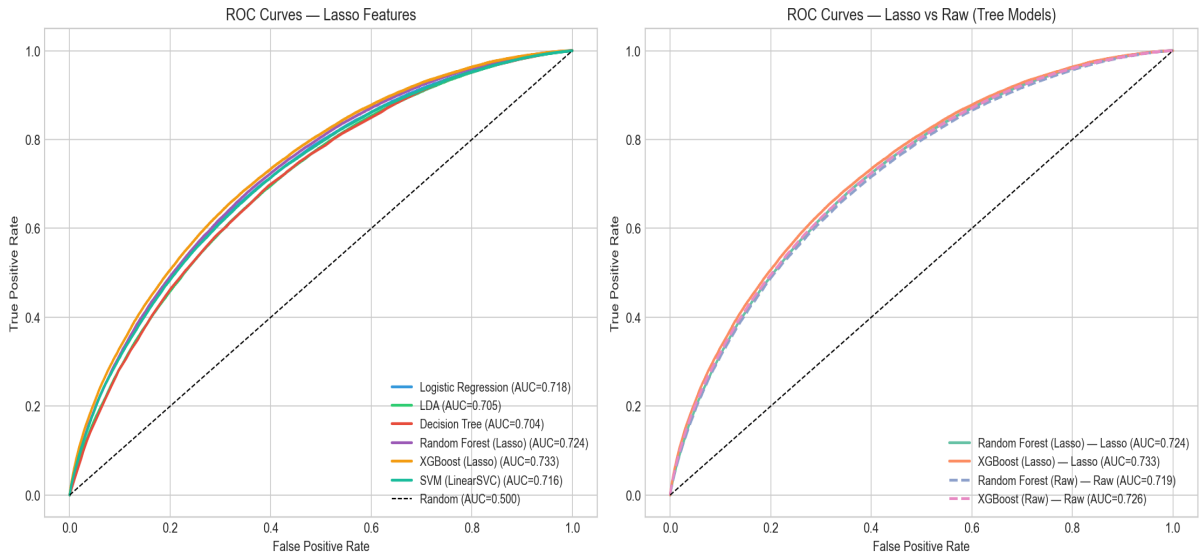


Figure 10: ROC Curves: Lasso Models (left) and Lasso vs Raw Tree Models (right)

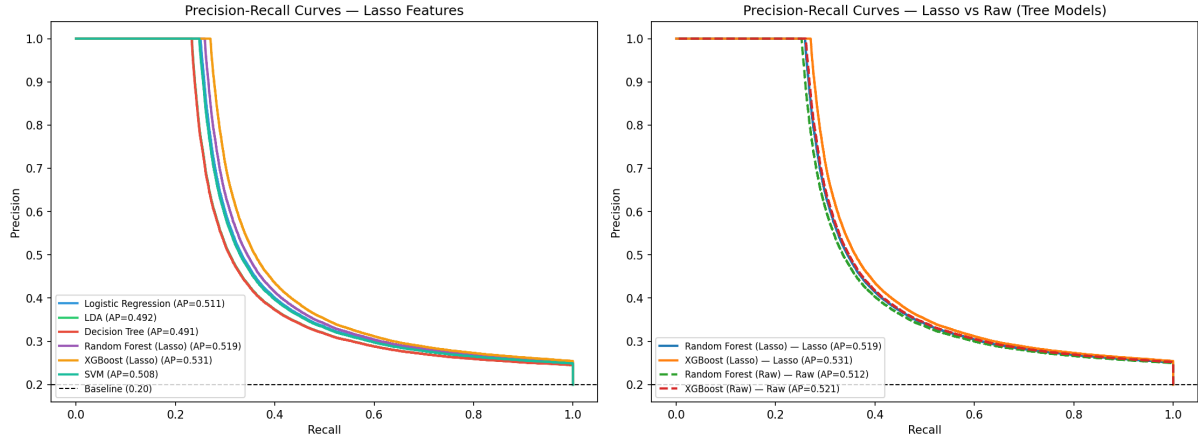


Figure 11: Precision-Recall Curves — Lasso Models (left) and Lasso vs Raw Tree Models (right). The dashed baseline represents the 0.20 default rate. Average Precision (AP) summarizes the curve, with XGBoost achieving the highest AP, confirming its superiority in catching defaults despite class imbalance.

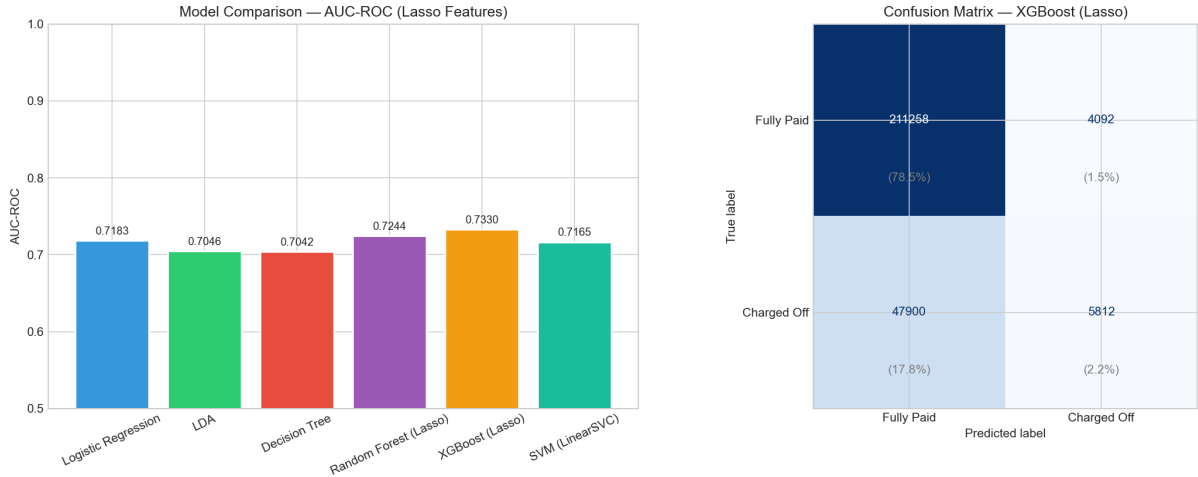


Figure 12: AUC-ROC Comparison (left) and XGBoost Confusion Matrix at $t=0.50$ (right)

7.3 Grade vs ML: Can We Beat LendingClub?

If we use only LendingClub’s grade to predict default, we get an AUC of 0.6801. XGBoost improves this to 0.7330, which is a 7.78% relative improvement. This is meaningful because it shows the model captures useful patterns beyond what the grade alone provides, likely by combining grade with loan purpose, verification status, and borrower behaviour metrics that the grade does not fully capture.

7.4 Threshold Optimisation

At the default threshold of 0.50, XGBoost only catches 10.8% of actual defaulters. In a real lending scenario, missing 9 out of 10 bad loans would be a serious problem. We tested every threshold from 0.01 to 0.99 and found that lowering it to 0.23 maximises the F1 score, bringing defaulter recall from 10.8% → 62.0%. Table 4 shows that every

model benefits substantially from lowering the threshold, but tree models improve the most because they tend to push most predictions toward 0 and 1, making the 0.50 almost useless for catching defaults.

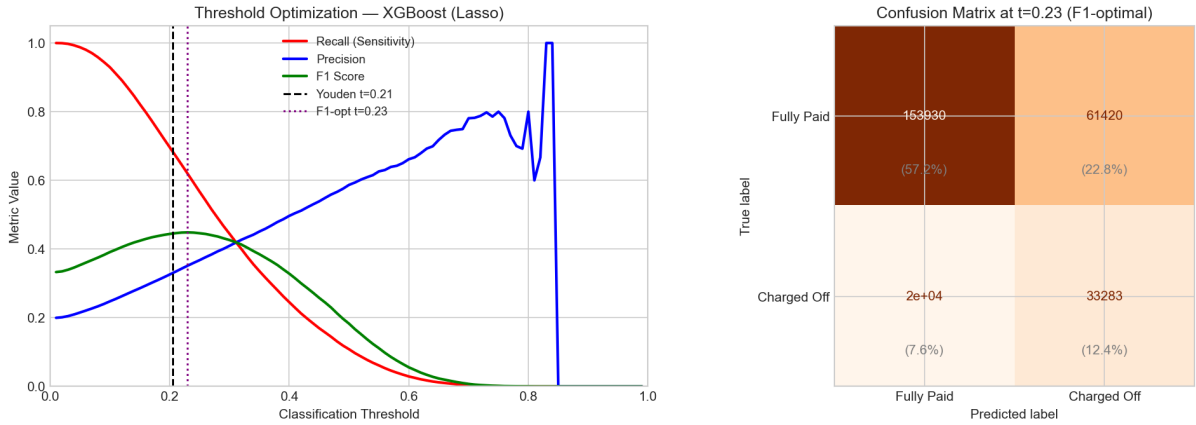


Figure 13: Threshold Sweep (left) and XGBoost Confusion Matrix at $t=0.23$ (right)

Table 4: Threshold Optimisation: Default ($t=0.50$) vs F1-Optimal

Model	Opt. t	Recall @0.50	Recall @Opt	F1 @Opt
Logistic Regression	0.35	0.327	0.620	0.438
LDA	0.34	0.294	0.622	0.424
Decision Tree	0.23	0.123	0.684	0.424
Random Forest	0.26	0.098	0.632	0.440
XGBoost	0.23	0.108	0.620	0.449
SVM	0.35	0.336	0.619	0.435

7.5 Feature Importance

Figure 14 ranks the top 20 features by how much each one improves XGBoost’s splitting decisions. LendingClub’s grade comes first at 11.1%, followed by verification status at 16.5% combined and debt consolidation purpose at 8.8%. Interestingly, `term` appears at number six with 6.0%, confirming that loan term is one of the strongest risk predictors in the dataset. The remaining features in the list cover mortgage history, recent inquiries, and revolving credit behaviour, which add real predictive power beyond what the grade alone provides.

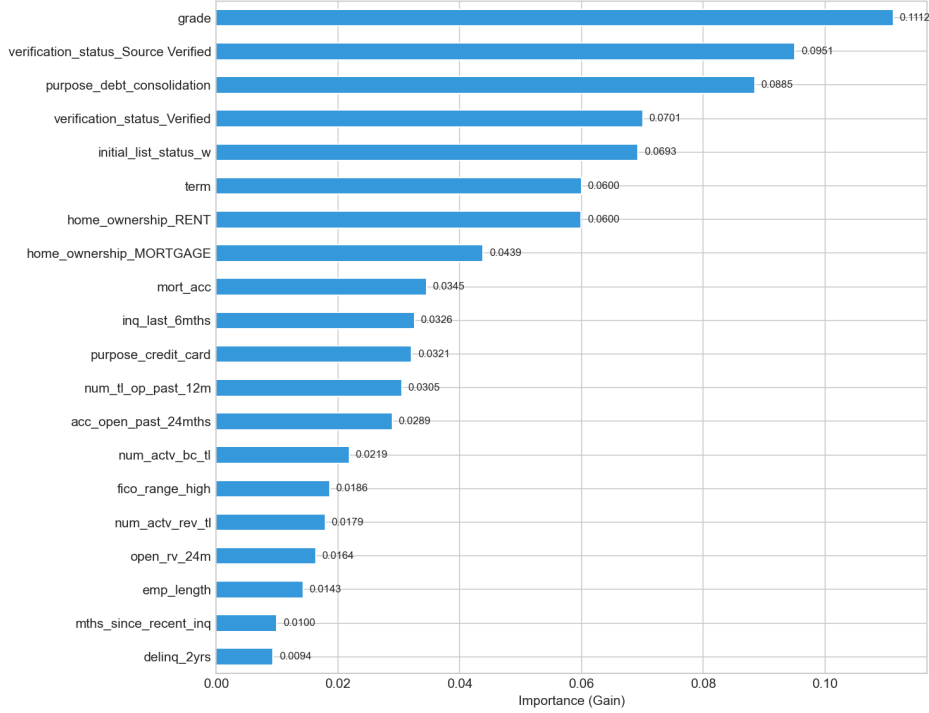


Figure 14: XGBoost Feature Importance (Top 20, Gain)

8 Portfolio Risk Profiling & Active Loan Scoring

8.1 Risk Tier Validation on Test Set

To translate model probabilities into business rules, we divided the test set into five equal risk tiers (quintiles) based on predicted default probability. Table 5 shows a clear separation: the “Very High” tier defaults at 41.9%, which is $8.5\times$ higher than the overall average. Crucially, the top two tiers capture 67.0% of all defaults while representing only 40.0% of the loan volume. This proves the model ranks risk effectively and establishes a precedent for focusing interventions on the riskiest 40% of the portfolio.

Table 5: Risk Tier Summary (Test Set, Quintile Partition)

Tier	Count	% of Loans	Avg. Prob	Default Rate	Cum. Defaults
Very Low	53,813	20.0%	0.054	4.9%	4.9%
Low	53,812	20.0%	0.114	10.9%	15.9%
Medium	53,812	20.0%	0.175	17.1%	33.0%
High	53,812	20.0%	0.256	25.0%	58.0%
Very High	53,813	20.0%	0.420	41.9%	100.0%

8.2 Active Loan Risk Profiling

Having validated the tier system, the XGBoost pipeline scored 912,569 active loans (Current/Late status). Figure 15 shows the predicted default probability distribution. The mean predicted probability is 0.246 (median 0.219), with 32.1% (293,259 loans) classified

as high-risk (prob > 0.30) and 7.2% (65,408 loans) as very high-risk (prob > 0.50). Applying the 40% volume heuristic from the test set validation, these high-risk active loans represent potential future losses requiring immediate, targeted monitoring rather than broad portfolio interventions.

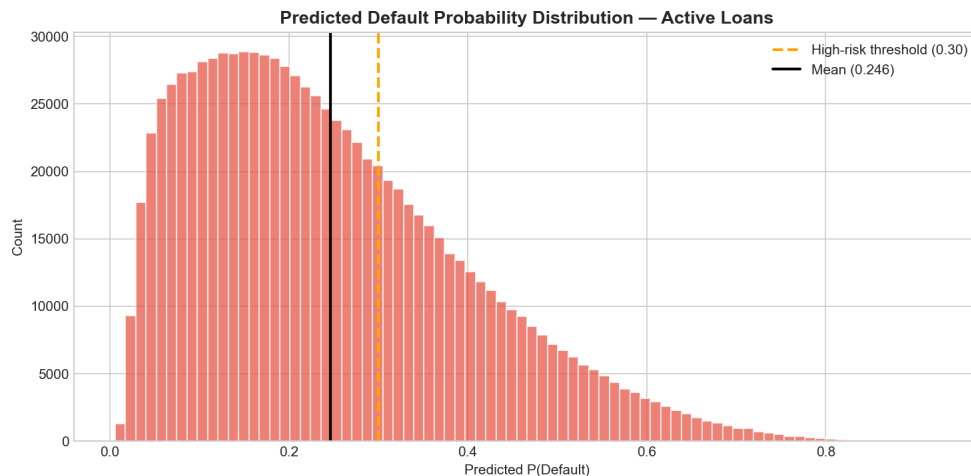


Figure 15: Predicted Default Probability Distribution — 912,569 Active Loans

9 Actionable Insights

1. **The alert threshold is the single most impactful operational decision.** At the default threshold of 0.50, XGBoost catches only 10.8% of actual defaulters. Lowering it to 0.23, the F1-optimal value, raises recall to 62.0%, catching 6 in 10 bad loans instead of 1 in 10. Precision drops from 58.7% to 35.1%, meaning more good borrowers are also flagged. However, in lending, approving a loan that defaults is typically 5 to 10 times more costly than declining a borrower who would have repaid. This single parameter change delivers the fastest reduction in portfolio losses with no new data and no new model.

To illustrate the financial impact, consider a portfolio where the average charged-off loan loses \$8,000 (a rough midpoint given the dataset’s loan range). At the default threshold of 0.50, the model catches 5,812 defaulters in the test set. Lowering the threshold to 0.23 raises this to 33,283 caught defaulters, an increase of 27,471 loans. At \$8,000 average loss per default, this represents around \$220 million in prevented losses on the test set alone, scaled proportionally to the full portfolio. The cost of false positives, good borrowers incorrectly flagged, rises too, but since a missed default typically costs five to ten times more than a wrongly declined good borrower, the net benefit of lowering the threshold remains strongly positive.

2. **Small business loans carry the highest default risk and need separate underwriting.** Small business loans default at nearly 28%, the highest of any loan purpose. This risk is not captured by LendingClub’s internal grade: Lasso assigned `purpose_small_business` the second largest coefficient at 0.058, confirming it adds strong predictive signal beyond the grade. These borrowers are being systematically underpriced. Requiring business financial documentation, capping loan amounts, or

applying stricter debt-to-income limits for this category would reduce losses without broadly restricting credit elsewhere.

3. **Adjust interest rates for low FICO scores instead of rejecting them outright.** Figure 4 shows that people who default usually have FICO scores about 30 points lower. However, simply rejecting everyone below a certain score is a bad idea. Our model shows that a borrower with a 650 score and very low debt can actually be safer than someone with a 680 score and maxed-out credit cards. Instead of rejecting these lower-scoring borrowers, we should use the model to slightly raise their interest rates based on their specific risk profile. This keeps the business growing by approving more loans, while still charging a fair price for the actual risk.
4. **Unverified income is a hidden risk factor that grows worse at lower grades.** Among Grade D borrowers, unverified income loans default at 31% versus 27% for verified ones, a 4 percentage point gap. This widens sharply at Grade G, where unverified borrowers default at 50%, meaning one in every two loans is lost. Requiring income verification for all applicants below Grade C would resolve this problem at minimal operational cost.
5. **Limit exposure for long-term loans and high-debt borrowers.** Our model found that 60-month loans and borrowers with a debt-to-income (DTI) ratio above 35% are the strongest predictors of default. LendingClub already charges higher interest rates for both groups. However, our data shows these extra charges are not enough to cover the actual losses. Simply raising rates even higher is a bad idea because it will only attract desperate borrowers. Instead, the platform should put strict limits on how much money these groups can borrow and require proof of income for long-term loans. This reduces financial exposure without losing them as customers entirely.
6. **The machine learning model catches risks that LendingClub’s grade misses, and works best as a second check.** Using grade alone gives an AUC of 0.6801. XGBoost reaches 0.7330, a relative improvement of 7.78%, by combining grade with verification status, loan purpose, and behavioural credit metrics that the grade does not fully capture. The most effective operational use is to flag loans where the model’s risk score disagrees significantly with the assigned grade, for example a Grade B loan scored as high-risk, and route those cases to review manually by a human before approval.
In practice, the disagreement check works as follows. Each loan is assigned a risk score between 0 and 1 by the model, then placed into a tier matching the grade scale: scores below 0.10 correspond to Grade A risk, 0.10 to 0.20 to Grade B, 0.20 to 0.30 to Grade C, 0.30 to 0.40 to Grade D, and above 0.40 to Grade E or below. A loan is flagged for manual review when the model tier is two or more steps below the assigned grade, for example a Grade B loan scoring above 0.30, which places it in Grade D territory.
7. **About a third of current loans are high-risk.** Out of 912,000 active loans, our model flagged 293,000 (32%) as likely to default, including 65,000 where a loss is more likely than being repaid. By focusing on just the riskiest 40% of loans, we can catch 67% of all future defaults. Contacting these borrowers early to offer flexible payment plans will save more money than just waiting for them to miss payments.

A rough estimate: Let's say contacting these borrowers costs about \$4.9 million (\$10 per high-risk, \$30 per very-high-risk), but preventing even 10% of their defaults saves roughly \$78 million at \$8,000 loss per loan. That is a \$73 million net gain, making outreach clearly worth the cost.

10 Conclusion & Limitations

This analysis built a complete ML pipeline starting from 2.26 million raw records, reducing to 77 Lasso-selected features, with SMOTE applied correctly inside each cross-validation fold. The final XGBoost model achieved an AUC of 0.7330, a 7.8% improvement over LendingClub's internal grade alone. The most important finding is that the deployment threshold, not the choice of model, is the single most impactful operational decision: lowering the threshold from 0.50 to 0.23 raises defaulter recall from 10.8% to 62.0% with no retraining required.

Limitations:

- **Survivorship bias:** The model was trained only on loans that LendingClub already approved, so it should not be applied directly to new applications without accounting for this selection effect.
- **Random split instead of time-based split:** The 80/20 split mixes loans from different economic periods between 2007 and 2018. Training on earlier years and testing on later years would give a more realistic picture of how the model performs over time.
- **SVM trained on a smaller sample:** SVM was trained on only 50,000 rows due to computational limits, so its AUC of 0.7165 is not directly comparable to the other models trained on the full dataset.
- **LDA works under assumptions that do not fully hold here:** LDA expects each class to follow a bell-shaped distribution, which is partially violated by the skewed financial variables and binary columns in this dataset. LDA was therefore trained on numeric features only, though the effect on AUC was small at this scale.

References

- [1] wordsforthewise (2018). *LendingClub Loan Data* [Dataset]. Kaggle. <https://www.kaggle.com/datasets/wordsforthewise/lending-club>